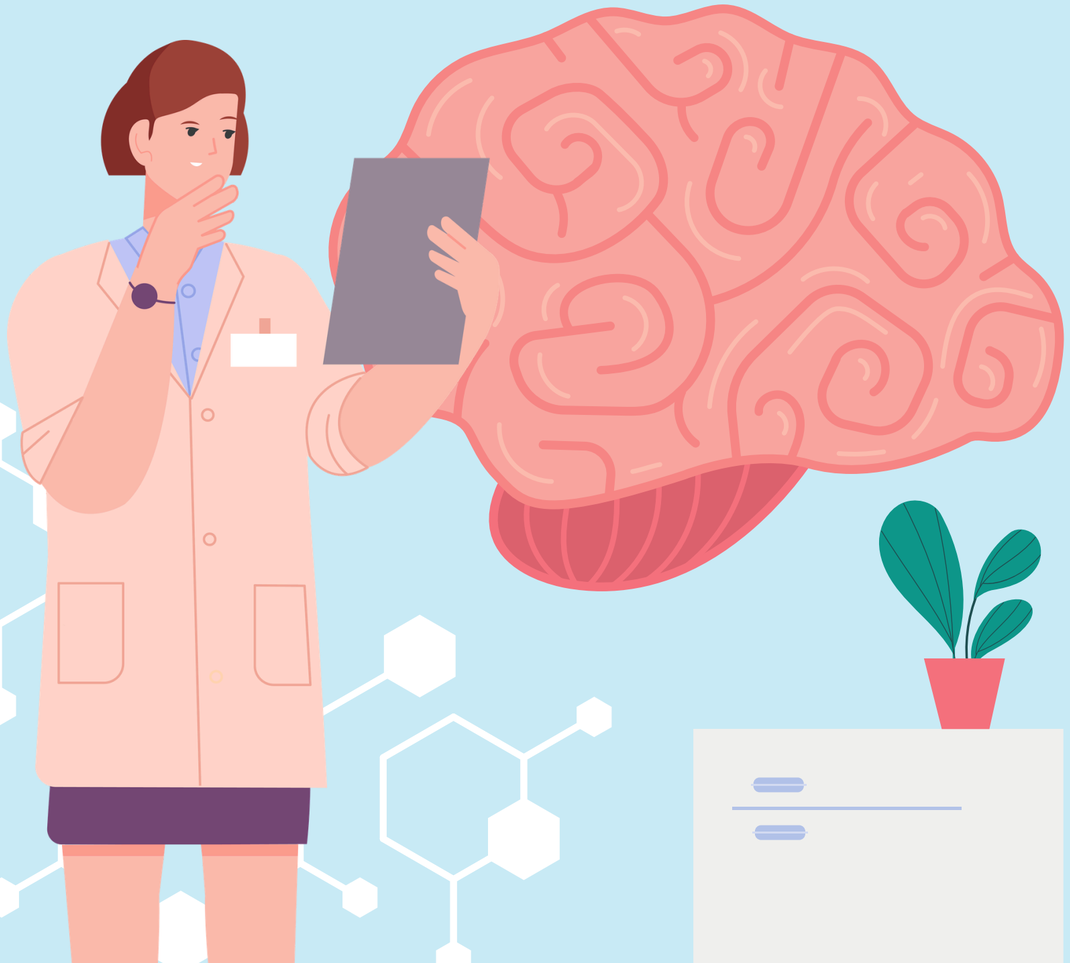
Computer Vision

Klasifikasi Citra Tumor Otak menggunakan Hybrid Feature Extraction dan SVM

Fadhli Ilham Nafi'an Yuswono
(2318001)

Latar Belakang dan Tujuan



1

Problem

Deteksi tumor otak secara manual dari citra MRI membutuhkan ketelitian tinggi dan rentan terhadap human error

2

Solution

Membangun model Machine Learning dengan algoritma SVM untuk mengklasifikasikan citra MRI menjadi kelas Tumor dan Sehat

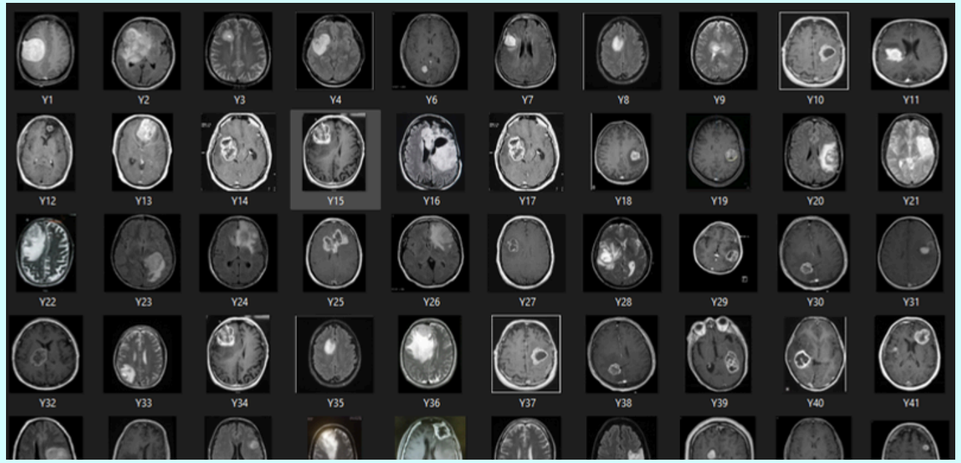
3

Approach

Menggabungkan 3 jenis fitur yaitu warna, bentuk, dan tekstur untuk mendapatkan akurasi yang lebih optimal

Dataset Overview

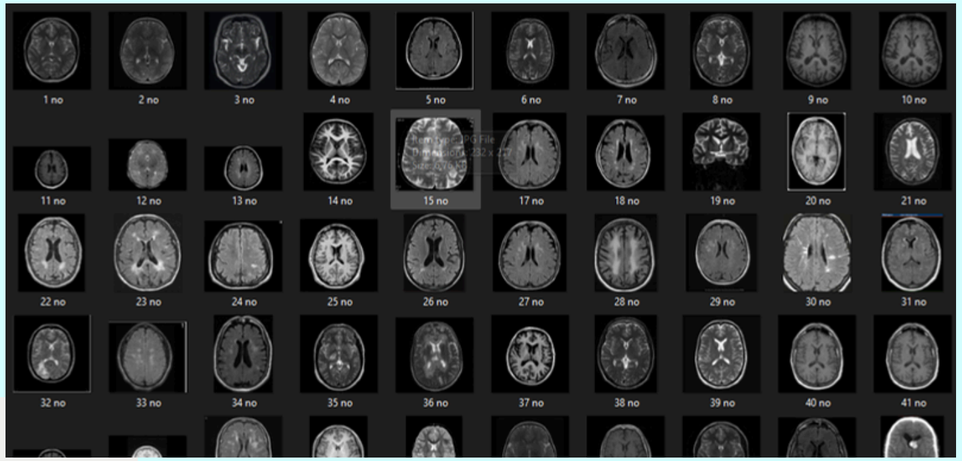
Dataset Sekunder "Brain Tumor" Jakesh Bohaju :
https://www.kaggle.com/datasets/navoneel/brain-mri-images-for-brain-tumor-detection



Data : 253 sampel citra MRI yang dikelompokkan ke dalam dua kategori yaitu 'yes' / terdapat tumor dan 'no' untuk otak sehat/normal

YES!

NO!



Import Library & Computational Tools

1 Import Library & Koneksi Dataset

```python
import os
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from skimage.feature import graycomatrix, graycoprops
from skimage.measure import shannon_entropy
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_curve, auc
from google.colab import drive

plt.style.use('seaborn-v0_8-muted')
sns.set_theme(style="whitegrid")

drive.mount('/content/drive')
base_path = '/content/drive/MyDrive/Study/Semester 6/Computer Vision/UTS/Dataset/brain_tumor_dataset'
categories = ['no', 'yes']
print("✅ Drive berhasil tersambung!")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
✅ Drive berhasil tersambung!
```

- Pandas : u/ manajemen data frame dan memproses dataset tabular
- NumPy : mendukung komputasi matriks dan operasi array numerik yang efisien
- OpenCV (cv2) : library utama u/ manipulasi citra dan ekstraksi fitur kontur
- Scikit-Image : u/ ekstraksi fitur tekstur jaringan menggunakan matriks GLCM
- Scikit-Learn : engine utama u/ preprocessing (StandardScaler) dan permodelan SVM
- Matplotlib & Seaborn : u/ visualisasi distribusi data, kurva ROC, dan heatmap confusion matrix

Ekstraksi Fitur Warna & Tekstur

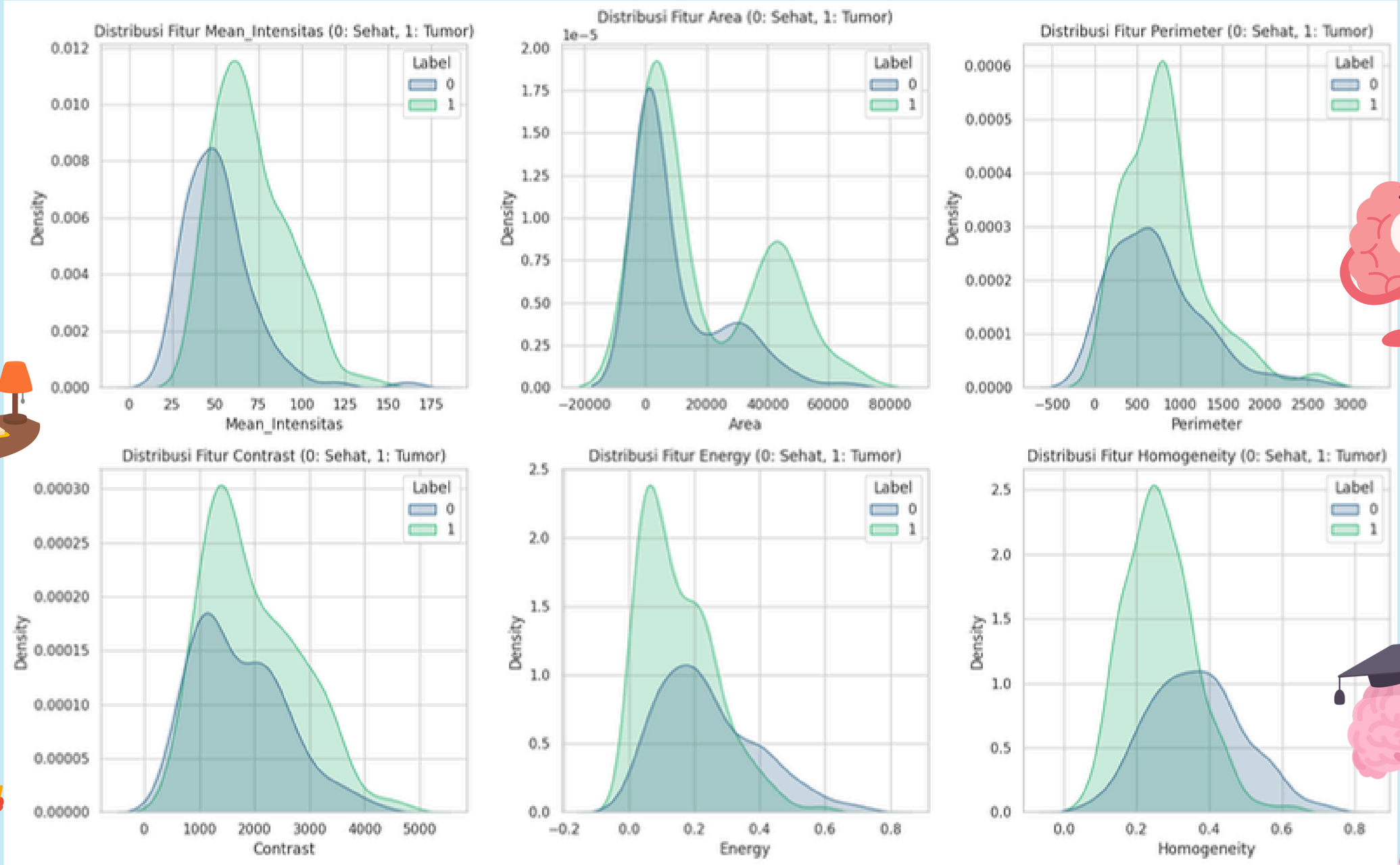
```python
data_features = []
labels = []
print("🔄 Memproses ekstraksi fitur Warna, Tekstur, dan Bentuk... Mohon tunggu.")

for category in categories:
    folder_path = os.path.join(base_path, category)
    label = categories.index(category)

    for file in os.listdir(folder_path):
        img_path = os.path.join(folder_path, file)
        img = cv2.imread(img_path, 0) # Grayscale

        if img is not None:
            img = cv2.resize(img, (256, 256))

            # Fitur warna
            mean_intensity = np.mean(img)
            std_intensity = np.std(img)

            # Fitur Tekstur (GLCM)
            glcm = graycomatrix(img, distances=[1], angles=[0], levels=256, symmetric=True, normed=True)
            contrast = graycoprops(glcm, 'contrast')[0, 0]
            energy = graycoprops(glcm, 'energy')[0, 0]
            homogeneity = graycoprops(glcm, 'homogeneity')[0, 0]

            # Fitur Bentuk (Morfologi Contours)
            # Thresholding untuk mencari bentuk massa putih u/ otak/tumor
            _, thresh = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
            contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

            area = 0
            perimeter = 0
            # Ambil contour terbesar
            if len(contours) > 0:
                cnt = max(contours, key=cv2.contourArea)
                area = cv2.contourArea(cnt)
                perimeter = cv2.arcLength(cnt, True)

            # Gabungkan ke dalam list
            data_features.append([mean_intensity, std_intensity, area, perimeter, contrast, energy, homogeneity])
            labels.append(label)

# Simpan ke DataFrame
kolom_fitur = ['Mean_Intensitas', 'Std_Intensitas', 'Area', 'Perimeter', 'Contrast', 'Energy', 'Homogeneity']
df = pd.DataFrame(data_features, columns=kolom_fitur)
df['Label'] = labels

print(f"✅ Berhasil mengekstraksi {len(df)} citra dengan Kombinasi 7 Fitur!")
df.head()
```

- Mean Intensitas : menghitung rata-rata kecerahan piksel citra MRI
- Std Intensitas : menghitung variasi sebaran tingkat kecerahan piksel
- Contrast : mengukur intensitas perbedaan lokal antar piksel / kekasaran jaringan
- Energy : mengukur tingkat keseragaman distribusi intensitas piksel
- Homogeneity : mengukur kedekatan distribusi elemen dalam matriks GLCM
- Thresholding : memisahkan objek massa putih (tumor/otak) dari latar belakang gelap
- Find Contours : mendeteksi garis tepi luar objek untuk isolasi massa utama
- Area : menghitung luas total piksel yang membentuk massa objek terbesar
- Perimeter : menghitung panjang keliling atau batas luar wilayah objek

```
🔄 Memproses ekstraksi fitur Warna, Tekstur, dan Bentuk... Mohon tunggu.
✅ Berhasil mengekstraksi 253 citra dengan Kombinasi 7 Fitur!
```

	Mean_Intensitas	Std_Intensitas	Area	Perimeter	Contrast	Energy	Homogeneity	Label
0	30.529587	47.663105	24466.0	650.558436	2094.551544	0.387339	0.545106	0
1	63.135803	47.426645	310.0	328.391916	1154.247463	0.169453	0.283244	0
2	35.038589	46.143573	1433.5	1073.075244	1618.753797	0.452405	0.489674	0
3	49.114792	42.415745	32.5	42.041630	568.596520	0.068616	0.221826	0
4	76.433258	85.595387	33625.5	696.683327	3494.752117	0.394766	0.441173	0

Exploratory Data Analysis (EDA)



Preprocessing & Training SVM

```
X = df.drop('Label', axis=1)
y = df['Label']

# Split data 80% Train, 20% Test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standarisasi data
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("✅ Data telah dibagi dan di-scaling!")
```

```
✅ Data telah dibagi dan di-scaling!
```

Train Test Split : membagi dataset menjadi 80% untuk training dan 20% untuk testing

StandardScaler : menyamakan skala fitur. misal Area vs Energy, dll. agar tidak terjadi dominasi bias pada model

```
# Menggunakan Kernel Linear untuk klasifikasi optimal
svm_model = SVC(kernel='linear', probability=True, random_state=42)
svm_model.fit(X_train_scaled, y_train)

y_pred = svm_model.predict(X_test_scaled)
print(f"🎯 Akurasi Model: {accuracy_score(y_test, y_pred)*100:.2f}%")
```

```
🎯 Akurasi Model: 76.47%
```

SVC Linear : mencari garis batas *hyperplane* paling optimal u/ memisahkan data medis yang banyak fitur

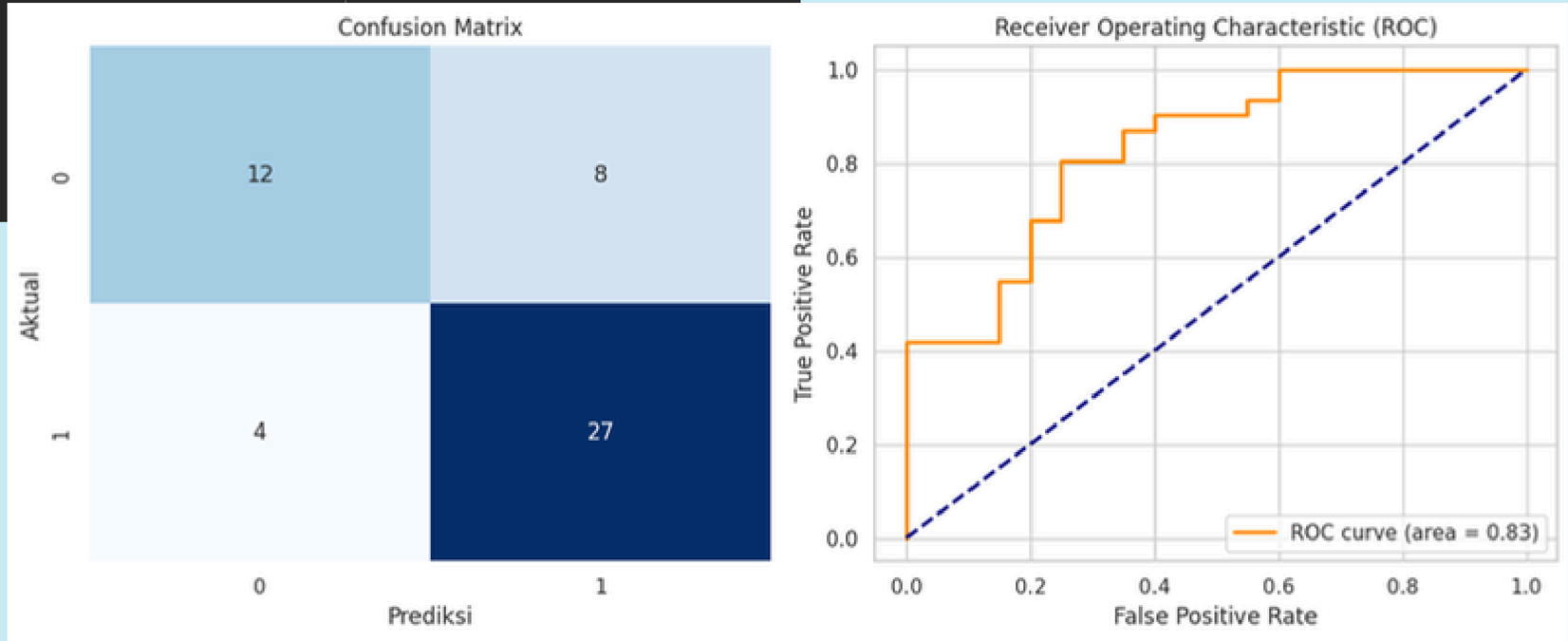
Evaluasi Performa Model

```python
# 1. Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.title('Confusion Matrix')
plt.xlabel('Prediksi')
plt.ylabel('Aktual')

# 2. ROC Curve
y_score = svm_model.predict_proba(X_test_scaled)[:,1]
fpr, tpr, _ = roc_curve(y_test, y_score)
roc_auc = auc(fpr, tpr)

plt.subplot(1, 2, 2)
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.title('Receiver Operating Characteristic (ROC)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")

plt.tight_layout()
plt.show()
```

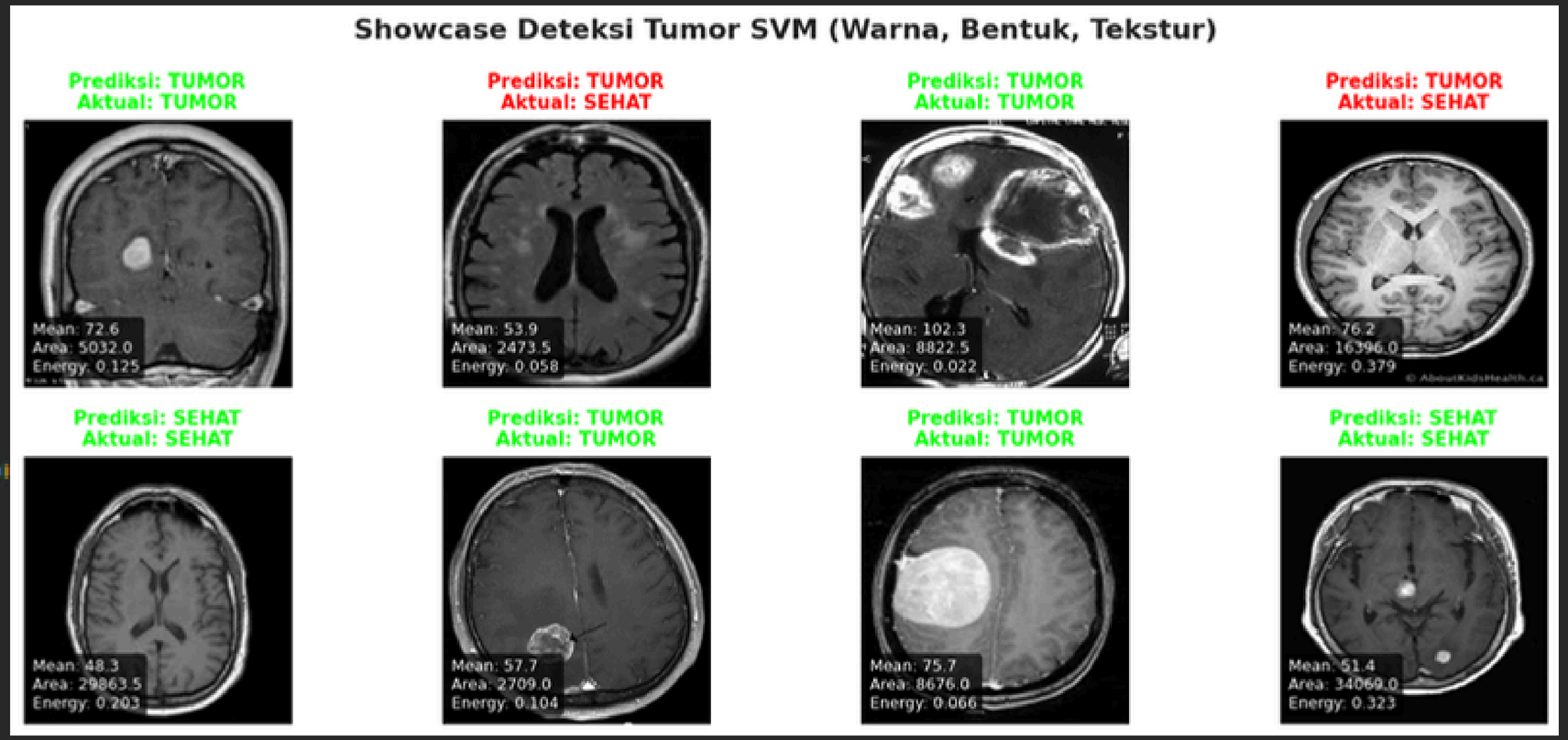
Analisis : model berhasil memprediksi 27 data tumor secara tepat dan 12 data sehat secara tepat

Error Rate : menekan tingkat kesalahan prediksi / False Negative yang sangat krusial dalam domain medis

ROC Curve : mendapatkan nilai AUC sebesar 0,83 yang menandakan kemampuan pemisahan kelas yang baik



Showcase Deteksi Tumor



- Preview : menampilkan 8 sampel citra acak dari data uji dengan label Prediksi AI vs Label Aktual
- Color Coding : judul berwarna Hijau untuk prediksi yang benar dan Merah untuk prediksi yang meleset
- Data Validasi : menampilkan nilai cuplikan Mean, Area, dan Energy langsung pada citra untuk pembuktian tekstur

Kesimpulan

1. Kombinasi Fitur Hybrid : penggabungan fitur Warna, Bentuk, dan Tekstur terbukti lebih komprehensif dalam menangkap anomali jaringan tumor
2. Pentingnya Preprocessing : penggunaan StandardScaler sangat krusial karena adanya rentang nilai fitur yang sangat timpang
3. Akurasi Realistis : hasil 76,47% merupakan angka yang memuaskan dan jujur untuk klasifikasi medis tanpa proses pembersihan tulang tengkorak / skull stripping
4. Relevansi Model : membuktikan bahwa algoritma Machine Learning klasik tetap tangguh jika didukung dengan rekayasa fitur / Feature Engineering yang tepat

Sekian

Terima Kasih

Sampai jumpa dipembelajaran selanjutnyaa